OBJECT ORIENTED PROGRAMING

OOP

PRACTICALS

NAME: SYEDA FAIZA ASLAM

SECTION: B

ROLL NO: CT-25064

LAB NO: 08

COURSE CODE: CT-260

COURSE TEACHER: Mr. AHMED ZAKI

Exercise

QUESTION: 1

Implement the given UML class diagram by following the guidelines given below.

- All the values are required to be set through parameterized constructor
- Provide necessary accessor methods where required.
- Create an object of the class bus by initializing it through a parameterized constructor in the main function and display all data members by calling the display function of class bus.

SOURCECODE:

```
#include <iostream>
using namespace std;

// Base Class
class Vehicle {
protected:
   string type;
   string make;
   string model;
   string color;
   int year;
   int miles;

public:
   Vehicle(string t, string mk, string md, string c, int y, int mi) {
      type = t;
      make = mk;
```

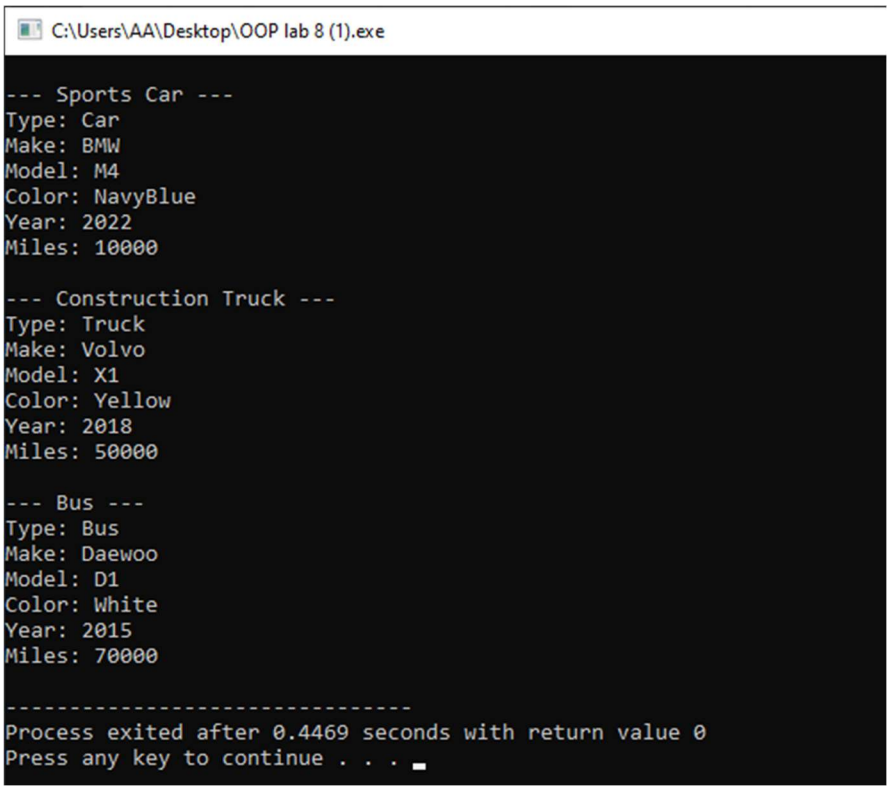
```
        model = md;
        color = c;
        year = y;
        miles = mi;
    }

    void display() {
        cout << "Type: " << type << endl;
        cout << "Make: " << make << endl;
        cout << "Model: " << model << endl;
        cout << "Color: " << color << endl;
        cout << "Year: " << year << endl;
        cout << "Miles: " << miles << endl;
    }
};

// Gas Vehicle
class GasVehicle : public Vehicle {
protected:
    int fuelCapacity;

public:
    GasVehicle(string t, string mk, string md, string c, int y, int mi, int f)
        : Vehicle(t, mk, md, c, y, mi) {
        fuelCapacity = f;
    }
};

// Electric Vehicle
class ElectricVehicle : public Vehicle {
protected:
    int batteryLife;

public:
    ElectricVehicle(string t, string mk, string md, string c, int y, int mi, int b)
```

```
        : Vehicle(t, mk, md, c, y, mi) {
        batteryLife = b;
    }
};

// High Performance
class HighPerformance : public GasVehicle {
protected:
    int horsepower;
    int topSpeed;

public:
    HighPerformance(string t, string mk, string md, string c, int y, int mi, int f,
                    int hp, int ts)
        : GasVehicle(t, mk, md, c, y, mi, f) {
        horsepower = hp;
        topSpeed = ts;
    }
};

// Sports Car
class SportsCar : public HighPerformance {
public:
    SportsCar(string t, string mk, string md, string c, int y, int mi, int f,
              int hp, int ts)
        : HighPerformance(t, mk, md, c, y, mi, f, hp, ts) {}

    void display() {
        cout << "\n--- Sports Car ---\n";
        Vehicle::display();
    }
};

// Heavy Vehicle
class HeavyVehicle : public GasVehicle {
```

```
protected:
   int weight;
   int wheels;

public:
   HeavyVehicle(string t, string mk, string md, string c, int y, int mi, int f,
            int w, int wh)
      : GasVehicle(t, mk, md, c, y, mi, f) {
      weight = w;
      wheels = wh;
   }
};

// Construction Truck
class ConstructionTruck : public HeavyVehicle {
public:
   ConstructionTruck(string t, string mk, string md, string c, int y, int mi,
                int f, int w, int wh)
      : HeavyVehicle(t, mk, md, c, y, mi, f, w, wh) {}

   void display() {
      cout << "\n--- Construction Truck ---\n";
      Vehicle::display();
   }
};

// Bus
class Bus : public HeavyVehicle {
public:
   Bus(string t, string mk, string md, string c, int y, int mi,
      int f, int w, int wh)
      : HeavyVehicle(t, mk, md, c, y, mi, f, w, wh) {}

   void display() {
      cout << "\n--- Bus ---\n";
```

```
        Vehicle::display();
    }
};

// Main
int main() {
    SportsCar s("Car", "BMW", "M4", "NavyBlue", 2022, 10000, 50, 400, 300);
    ConstructionTruck t("Truck", "Volvo", "X1", "Yellow", 2018, 50000, 120, 8000,
8);
    Bus b("Bus", "Daewoo", "D1", "White", 2015, 70000, 150, 10000, 6);

    s.display();
    t.display();
    b.display();

    return 0;
}
```

OUTPUT:



QUESTION: 2

Suppose you are designing a game engine for a new video game. The game engine needs to support different types of characters, such as warriors, mages, and archers. Each character type has specific attributes and abilities. The game also includes non-playable characters (NPCs) that have their own unique behaviors. Additionally, there are special characters called "Mighty" that possess both the abilities of a warrior and a mage. To design the character classes and utilize multiple inheritance in this scenario, you can create a class hierarchy with appropriate base classes and derived classes. Here's a possible implementation:

- Create a base class called Character that contains common attributes and behaviors for all characters, such as name, level, and health.
- Create derived classes for specific character types:
  a. Warrior class, derived from Character, which includes attributes and abilities specific to warriors, such as strength, melee weapons proficiency, and a "slash" ability.
  b. Mage class, derived from Character, which includes attributes and abilities specific to mages, such as intelligence, spell casting proficiency, and a "fireball" ability.
  c. Archer class, derived from Character, which includes attributes and abilities specific to archers, such as dexterity, ranged weapons proficiency, and a "rapid shot" ability.

- Create a class called NPC (Non-Playable Character), derived from Character, which includes additional behaviors specific to non-playable characters, such as predefined movement patterns or scripted dialogues.
- Create a class called Mighty, derived from both Warrior and Mage, to represent special Mighty characters. This class inherits attributes and abilities from both Warrior and Mage, allowing the Mighty to possess the combined traits of a warrior and a mage. For example, a Mighty might have high strength and melee prowess like a warrior, as well as the ability to cast powerful spells like a mage.

By using multiple inheritance, you can design a class hierarchy that allows for the sharing of common attributes and behaviors while also enabling specific character types to inherit and extend upon those features. The Mighty class demonstrates the ability to combine attributes and abilities from multiple base classes, showcasing the flexibility of multiple inheritance in this scenario.

SOURCECODE:

```
#include <iostream>
using namespace std;

// Base Class
class Character {
protected:
   string name;
   int level;
```

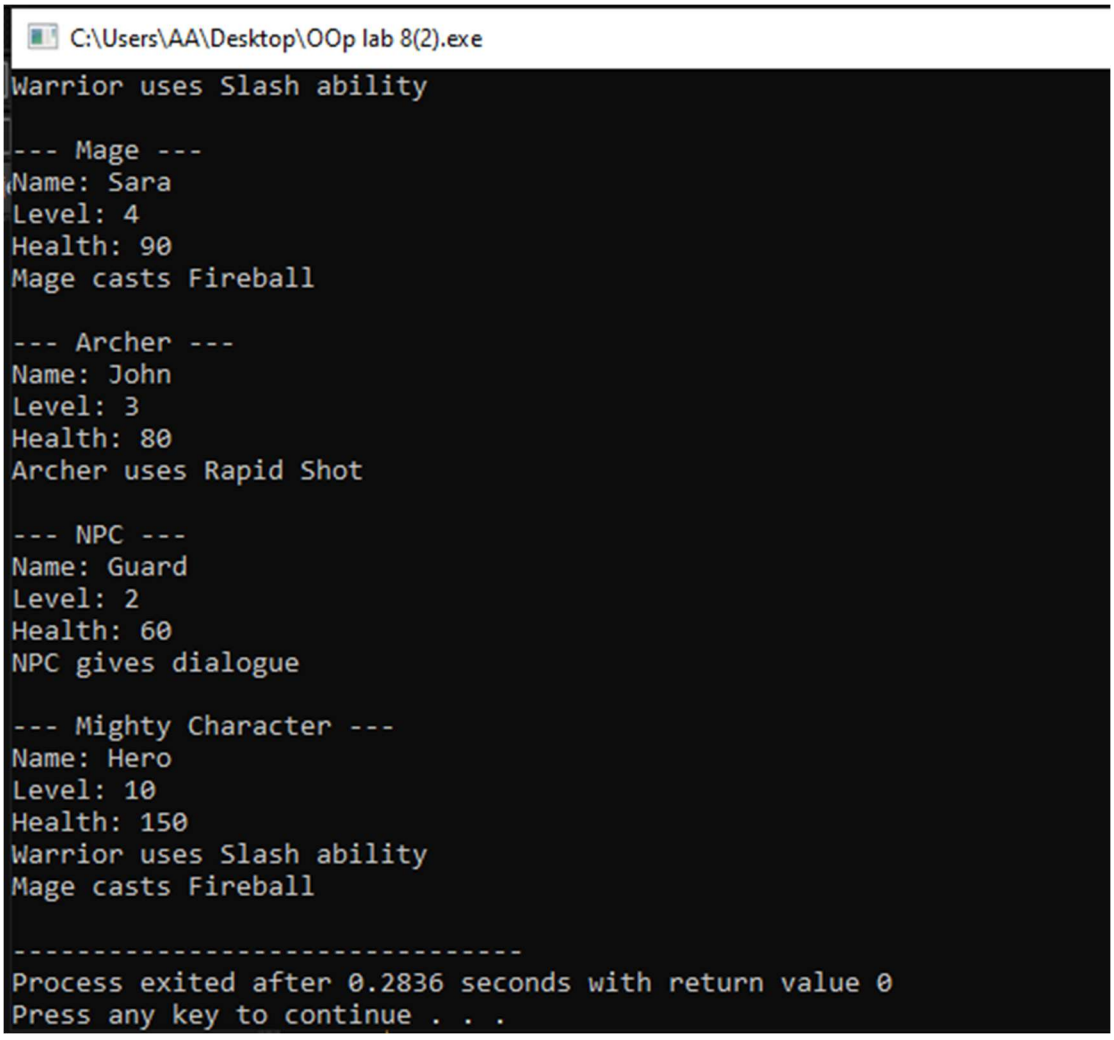
```
    int health;

public:
    Character(string n, int l, int h) {
        name = n;
        level = l;
        health = h;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Level: " << level << endl;
        cout << "Health: " << health << endl;
    }
};

// Warrior Class
class Warrior : public Character {
protected:
    int strength;

public:
    Warrior(string n, int l, int h, int s)
        : Character(n, l, h) {
        strength = s;
    }

    void slash() {
        cout << "Warrior uses Slash ability\n";
    }
};

// Mage Class
class Mage : public Character {
protected:
```

```
    int intelligence;

public:
    Mage(string n, int l, int h, int i)
        : Character(n, l, h) {
        intelligence = i;
    }

    void fireball() {
        cout << "Mage casts Fireball\n";
    }
};

// Archer Class
class Archer : public Character {
protected:
    int dexterity;

public:
    Archer(string n, int l, int h, int d)
        : Character(n, l, h) {
        dexterity = d;
    }

    void rapidShot() {
        cout << "Archer uses Rapid Shot\n";
    }
};

// NPC Class
class NPC : public Character {
public:
    NPC(string n, int l, int h)
        : Character(n, l, h) {}
```

```
    void talk() {
        cout << "NPC gives dialogue\n";
    }
};

// Mighty Class (Multiple Inheritance)
class Mighty : public Warrior, public Mage {
public:
    Mighty(string n, int l, int h, int s, int i)
        : Warrior(n, l, h, s), Mage(n, l, h, i) {}

    void displayMighty() {
        cout << "\n--- Mighty Character ---\n";
        Warrior::display();  // using one display
    }
};

// Main
int main() {
    Warrior w("Ali", 5, 100, 80);
    Mage m("Sara", 4, 90, 85);
    Archer a("John", 3, 80, 70);
    NPC npc("Guard", 2, 60);
    Mighty my("Hero", 10, 150, 95, 100);

    cout << "\n--- Warrior ---\n";
    w.display();
    w.slash();

    cout << "\n--- Mage ---\n";
    m.display();
    m.fireball();

    cout << "\n--- Archer ---\n";
    a.display();
```

```
    a.rapidShot();

    cout << "\n--- NPC ---\n";
    npc.display();
    npc.talk();

    my.displayMighty();
    my.slash();
    my.fireball();

    return 0;
}
```

OUTPUT:



QUESTION: 3

Implement the given UML class diagram by following the guidelines given below.

- The interest Account class adds interest for every deposit, assuming a default of 30%.
- The charging account class charges a default fee of Rs. 25 for every withdrawal.
- Transfer method of ACI class tasks two parameters: amount to be transferred and object of class in which we have to transfer that amount.
- Make parameterized constructor and default constructor to take user input for all data members.
- Make a driver program to test all functionalities.

SOURCECODE:

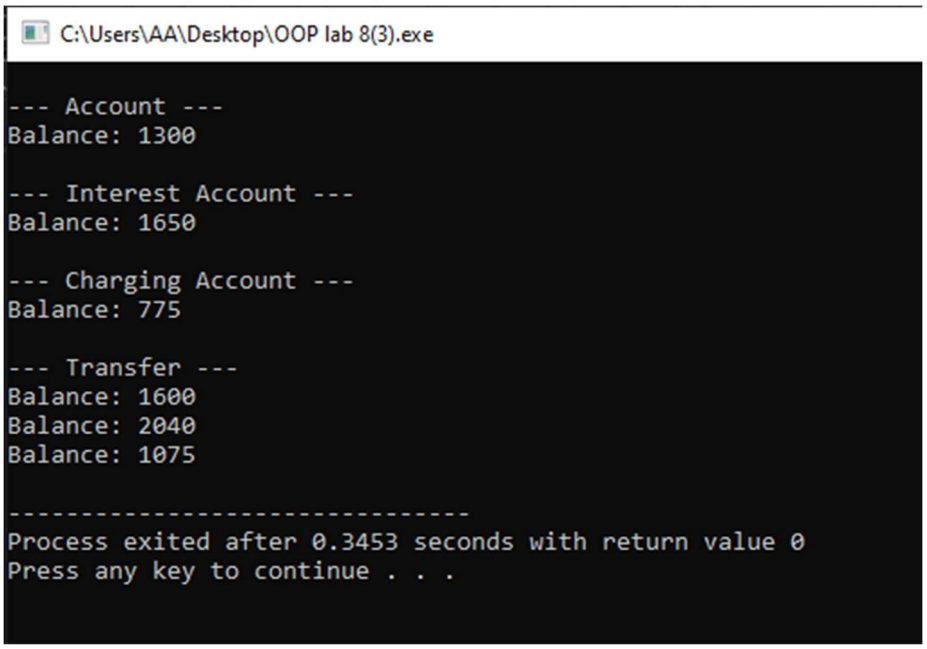
```
#include <iostream>
using namespace std;

// Base Class
class Account {
protected:
   double balance;

public:
   Account() {
      balance = 0;
   }

   Account(double b) {
      balance = b;
   }
```

```
    void deposit(double amount) {
        balance += amount;
    }

    void withdraw(double amount) {
        balance -= amount;
    }

    void checkBalance() {
        cout << "Balance: " << balance << endl;
    }
};

// Interest Account
class InterestAccount : public Account {
    double interest;

public:
    InterestAccount(double b) : Account(b) {
        interest = 0.30; // 30%
    }

    void deposit(double amount) {
        double bonus = amount * interest;
        balance += amount + bonus;
    }
};

// Charging Account
class ChargingAccount : public Account {
    double fee;

public:
    ChargingAccount(double b) : Account(b) {
```

```
        fee = 25;
    }

    void withdraw(double amount) {
        balance -= (amount + fee);
    }
};

// ACI Class
class ACI {
public:
    void transfer(double amount, Account &acc) {
        acc.deposit(amount);
    }

    void transfer(double amount, InterestAccount &acc) {
        acc.deposit(amount);
    }

    void transfer(double amount, ChargingAccount &acc) {
        acc.deposit(amount);
    }
};

// Main
int main() {
    Account a(1000);
    InterestAccount ia(1000);
    ChargingAccount ca(1000);

    ACI obj;

    cout << "\n--- Account ---\n";
    a.deposit(500);
    a.withdraw(200);
```

```
    a.checkBalance();

    cout << "\n--- Interest Account ---\n";
    ia.deposit(500);
    ia.checkBalance();

    cout << "\n--- Charging Account ---\n";
    ca.withdraw(200);
    ca.checkBalance();

    cout << "\n--- Transfer ---\n";
    obj.transfer(300, a);
    obj.transfer(300, ia);
    obj.transfer(300, ca);

    a.checkBalance();
    ia.checkBalance();
    ca.checkBalance();

    retur
```

OUTPUT:



QUESTION: 4

You are developing a software application for a library that manages various types of media, including books, magazines, and DVDs. Each type of media has specific attributes and functionalities, such as the author for books, issue number for magazines, and director for DVDs. Additionally, there are certain operations that are common to all types of media, such as borrowing, returning, and displaying information. Design a class hierarchy using multiple inheritance to handle the different types of media and their functionalities.

- Create a base class called Media to represent the common attributes and functionalities shared by all types of media. This class will include operations such as borrowing, returning, and displaying information.
- Create individual classes for each type of media, such as Book, Magazine, and DVD. Each class will inherit from both the Media class and their respective specific attribute classes.

With this class hierarchy, the library application can handle different types of media such as books, magazines, and DVDs. The common functionalities like borrowing, returning, and displaying information will be inherited from the Media class, while the specific attributes and functionalities for each media type will be defined in their respective classes.

SOURCECODE:

```
#include <iostream>
using namespace std;
```

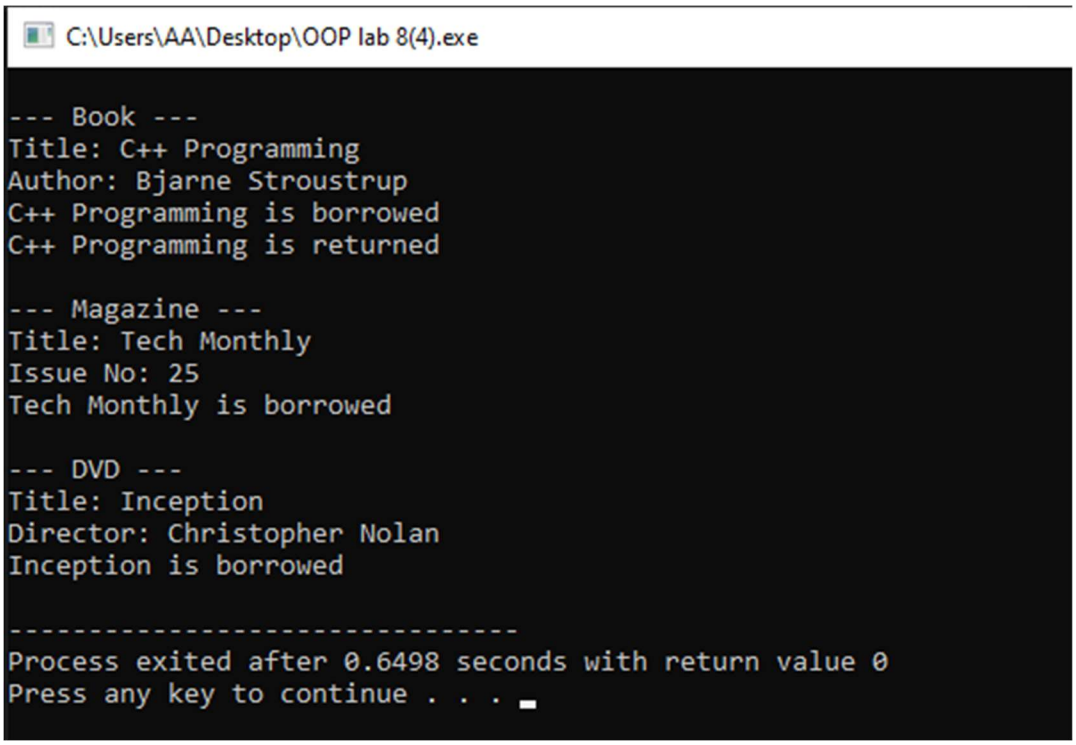
```
// Base Class
class Media {
protected:
   string title;

public:
   Media(string t) {
      title = t;
   }

   void borrow() {
      cout << title << " is borrowed\n";
   }

   void returnItem() {
      cout << title << " is returned\n";
   }

   void display() {
      cout << "Title: " << title << endl;
   }
};

// Book Specific Class
class BookInfo {
protected:
   string author;

public:
   BookInfo(string a) {
      author = a;
   }
};
```

```
// Magazine Specific Class
class MagazineInfo {
protected:
   int issueNo;

public:
   MagazineInfo(int i) {
      issueNo = i;
   }
};

// DVD Specific Class
class DVDInfo {
protected:
   string director;

public:
   DVDInfo(string d) {
      director = d;
   }
};

// Book Class
class Book : public Media, public BookInfo {
public:
   Book(string t, string a)
      : Media(t), BookInfo(a) {}

   void show() {
      cout << "\n--- Book ---\n";
      display();
      cout << "Author: " << author << endl;
   }
};
```

```
// Magazine Class
class Magazine : public Media, public MagazineInfo {
public:
   Magazine(string t, int i)
      : Media(t), MagazineInfo(i) {}

   void show() {
      cout << "\n--- Magazine ---\n";
      display();
      cout << "Issue No: " << issueNo << endl;
   }
};

// DVD Class
class DVD : public Media, public DVDInfo {
public:
   DVD(string t, string d)
      : Media(t), DVDInfo(d) {}

   void show() {
      cout << "\n--- DVD ---\n";
      display();
      cout << "Director: " << director << endl;
   }
};

// Main
int main() {
   Book b("C++ Programming", "Bjarne Stroustrup");
   Magazine m("Tech Monthly", 25);
   DVD d("Inception", "Christopher Nolan");

   b.show();
   b.borrow();
   b.returnItem();
```

```
    m.show();
    m.borrow();

    d.show();
    d.borrow();

    return 0;
}
```

OUTPUT:



SOLVED EXAMPLE

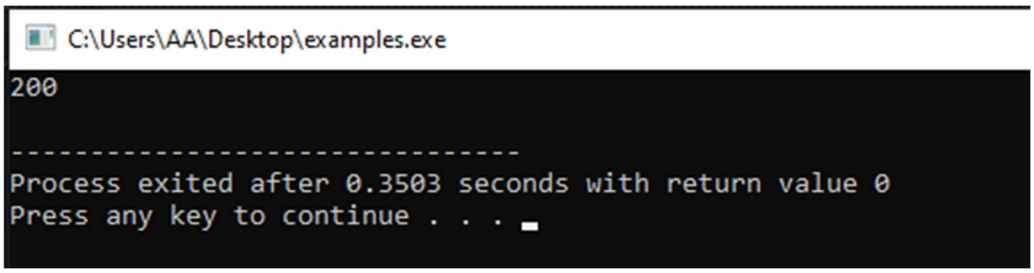
EXAMPLE: 1 (Diamond Problem (Error Case))

SOURCECODE:

```
#include <iostream>
using namespace std;

class A {
public:
   int a;
};

class B : public A {
public:
   int b;
};

class C : public A {
public:
   int c;
};

class D : public B, public C {
public:
   int d;
};

int main() {
   D obj;

   // obj.a = 200;  // ERROR (ambiguous)
```

```
    obj.B::a = 200;  // fix manually
    cout << obj.B::a << endl;

    return 0;
}
```

OUTPUT:



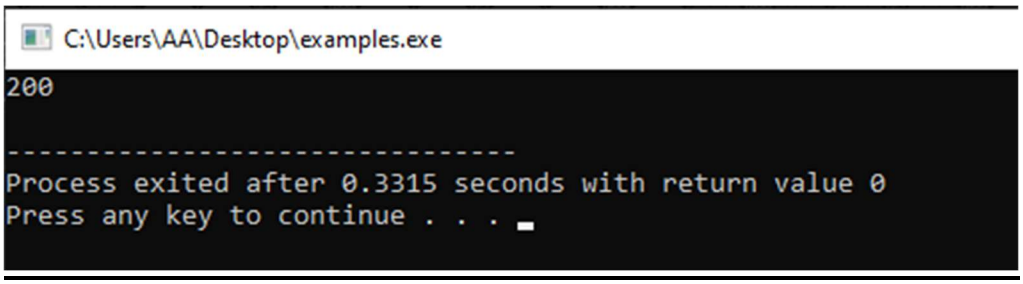
EXAMPLE: 2 (Solving Diamond Problem (Virtual Base Class))

SOURCECODE:

```
#include <iostream>
using namespace std;

class A {
public:
    int a;
};

class B : virtual public A {
public:
    int b;
};

class C : virtual public A {
public:
    int c;
```

```
};

class D : public B, public C {
public:
   int d;
};

int main() {
   D obj;

   obj.a = 200;  // no error now
   cout << obj.a << endl;

   return 0;
}
```

OUTPUT:



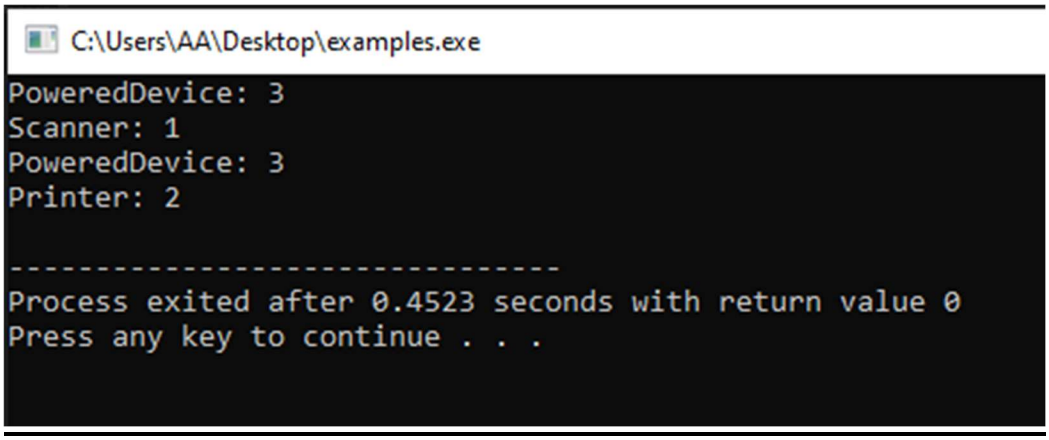
EXAMPLE: 3 (Real Example (Without Virtual - Problem)

SOURCECODE:

```
#include <iostream>
using namespace std;

class PoweredDevice {
public:
   PoweredDevice(int power) {
      cout << "PoweredDevice: " << power << endl;
   }
};
```

```
class Scanner : public PoweredDevice {
public:
   Scanner(int s, int p) : PoweredDevice(p) {
      cout << "Scanner: " << s << endl;
   }
};

class Printer : public PoweredDevice {
public:
   Printer(int pr, int p) : PoweredDevice(p) {
      cout << "Printer: " << pr << endl;
   }
};

class Copier : public Scanner, public Printer {
public:
   Copier(int s, int pr, int p)
      : Scanner(s, p), Printer(pr, p) {}
};

int main() {
   Copier c(1, 2, 3);
   return 0;
```

OUTPUT:



EXAMPLE: 4 Real Example (Solved with Virtual Base Class)

SOURCECODE:

```
#include <iostream>
using namespace std;

class PoweredDevice {
public:
   PoweredDevice(int power) {
      cout << "PoweredDevice: " << power << endl;
   }
};

class Scanner : virtual public PoweredDevice {
public:
   Scanner(int s, int p) : PoweredDevice(p) {
      cout << "Scanner: " << s << endl;
   }
};

class Printer : virtual public PoweredDevice {
public:
   Printer(int pr, int p) : PoweredDevice(p) {
      cout << "Printer: " << pr << endl;
   }
};

class Copier : public Scanner, public Printer {
public:
   Copier(int s, int pr, int p)
      : PoweredDevice(p), Scanner(s, p), Printer(pr, p) {}
};

int main() {
   Copier c(1, 2, 3);
```

```
    return 0;
}
```

OUTPUT: